[단국대학교]

AI융합대학 소프트웨어학과

CAPSTONE FESTIVAL 2026

기술 사양 문서

방토리

Bangtory

함께 사는 공간의 일상을 한 곳에서 관리하는 공동생활 관리 앱

팀명 방범대

팀원 32220630 김민영

32221595 민수연

32224802 최현경

32225007 홍수한

지도교수 박용범 교수님

1. 개요
2. 시스템 아키텍처
3. 백엔드 기술사양
3.1 기술 스택 개요
3.2 API 통신 및 설계 방식
3.3 보안 및 인증
3.4 데이터베이스 설계
3.5 실시간 기능
3.6 로그 및 에러 처리
3.7 배포 및 운영
3.8 프로젝트 구조
4. 프론트 기술사양
4.1 기술 스택 개요
4.2 화면 구성 및 주요 기능
4.3 상태 관리 방식
4.4 OAuth 인증 연동
4.5 알림 시스템
4.6 실시간 통신
4.7 디렉토리 구조 예시
5. 마무리

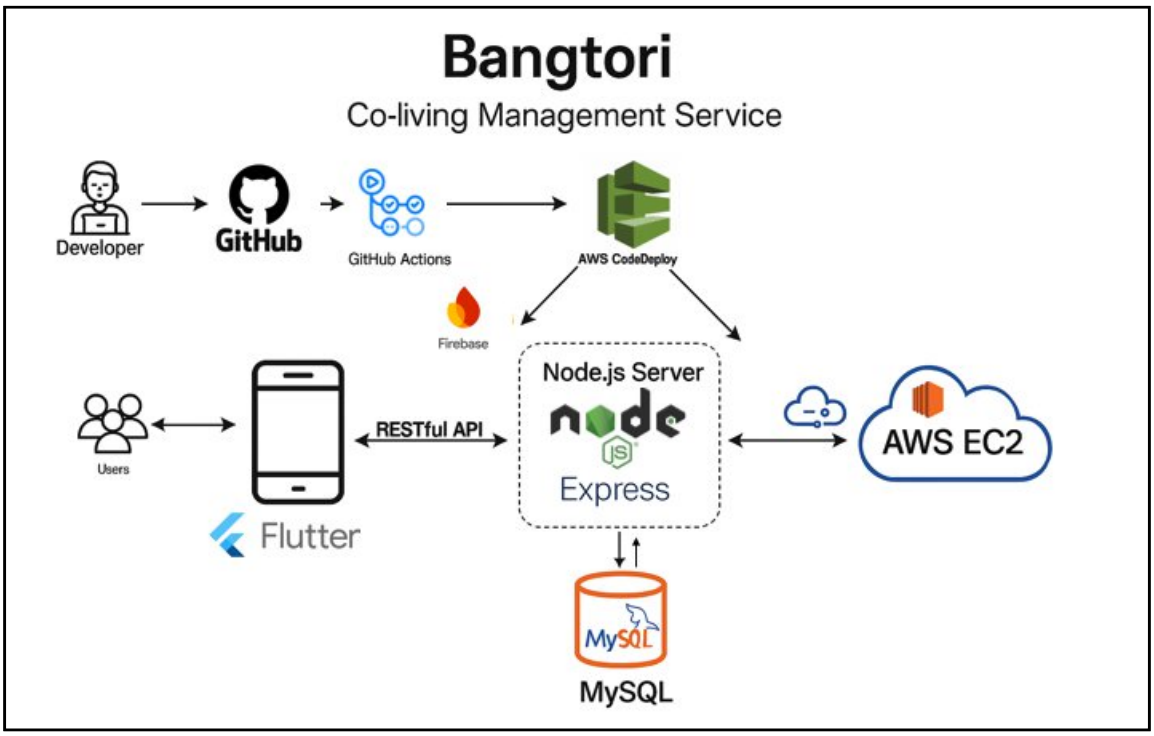
1 개요

본 기술 사양서는 Flutter 기반의 프론트엔드와 Node.js 기반의 백엔드로 구성된 룸메이트 관리 애플리케이션의 전체 기술 구조를 정의한 문서이다. 해당 시스템은 공동 일정 등록, 가전제품 예약, 방문객 예약, 실시간 채팅, 알림 수신, 상태 공유 등의 다양한 기능을 제공하며, Firebase와 MySQL을 병행 활용하여 실시간성과 안정성을 동시에 확보하고자 한다.

본 문서는 프로젝트 개발 과정에서의 일관된 기술 기준, 명확한 API 정의, 보안 원칙, 디렉토리 구조, 운영/배포 방식 등을 문서화함으로써, 팀원 간 협업과 유지보수의 효율성을 높이고자 한다.

2 시스템 아키텍처

이 프로젝트에서는 Flutter 기반의 클라이언트 애플리케이션**과** Node.js 기반의 백엔드 서버가 구성될 예정이며, AWS EC2 인스턴스 상에서 운영하고 GitHub Actions 및 AWS CodeDeploy를 통해 자동화된 배포 파이프라인을 구축하려고 한다.



개발자는 GitHub 저장소를 통해 소스코드를 관리하며, 코드 푸시 시 GitHub Actions가 자동으로 빌드 및 테스트를 수행한다. 이후 AWS CodeDeploy가 연동되어, 백엔드 서버가 배포되는 AWS EC2 인스턴스에 새로운 버전을 자동으로 배포한다. 이 과정을 통해 배포의 일관성과 안정성을 확보할 수 있다.

서버는 Node.js 기반의 Express 프레임워크 위에서 동작하며, 클라이언트인 Flutter 앱과는 RESTful API를 통해 통신한다. 사용자는 Flutter 앱을 통해 일정 등록, 채팅, 예약 등의 기능을 수행하고, 이에 대한 요청은 백엔드 서버에서 처리된다.

정형 데이터(회원 정보, 예약 등)는 MySQL에 저장되며, 비정형 데이터(실시간 채팅, 상태 공유 등)는 Firebase를 통해 처리된다. 푸시 알림은 Firebase Cloud Messaging(FCM)을 활용하여 사용자 단말기로 전달된다.

본 구조는 API 중심의 유연한 확장성과 CI/CD 기반의 운영 편의성을 동시에 고려한 설계이다.

3 백엔드 기술 사양

3.1 기술 스택 개요

본 프로젝트의 백엔드는 Node.js 런타임 환경에서 실행되며, Express.js를 기반으로 RESTful API 서버로 구축되어 있다. 실시간 기능과 푸시 알림, 인증, 데이터 저장을 위해 Firebase 플랫폼을 병행 사용하며, 정형 데이터 처리를 위해 MySQL을 도입하였다. 다음은 주요 기술 스택의 개요이다.

구분	기술 스택	설명
런타임	Node.js	비동기 처리를 위한 이벤트 기반 자바스크립트 런타임
프레임워크	Express.js	경량 웹 애플리케이션 프레임워크
API 문서화	Swagger (OpenAPI), Postman	API 정의 및 테스트 도구
인증	Firebase Authentication, JWT	사용자 인증 및 토큰 기반 세션 관리
실시간 기능	WebSocket, Firebase Firestore	채팅, 상태 동기화 등 실시간 처리
알림 시스템	Firebase Cloud Messaging (FCM)	푸시 알림 발송 및 수신
데이터베이스	MySQL, Firebase Firestore	정형/비정형 데이터 분산 저장
배포	AWS EC2, Firebase Hosting	백엔드 및 정적 파일 배포
환경 관리	dotenv	민감 정보 및 환경변수 분리 관리
보안	Helmet, CORS, express-validator	보안 헤더 설정, 입력 검증, CORS 필터링
로깅	Winston	에러 및 이벤트 로그 기록 및 관리

3.2 API 통신 및 설계 방식

백엔드는 RESTful API 설계 원칙을 기반으로 URL, 메서드, 응답 코드 체계를 일관되게 유지한다. 리소스 중심의 URI 설계 방식을 따르며, 다음과 같은 응답 형식을 따른다:

```
{
  "code": 200,
  "message": "성공",
  "data": { ... }
}
```

Swagger를 통해 전체 API 명세를 자동 문서화하며, Postman을 활용하여 API 테스트 및 팀 내 공유를 병행한다.

3.3 보안 및 인증

사용자 인증은 Firebase Authentication을 기반으로 한다. API 접근 제어는 다음과 같은 보안 조치를 통해 이뤄진다:

- **Helmet.js**: HTTP 보안 헤더를 통한 공격 방지 (예: XSS, CSP 등)
- **CORS 설정**: 허용된 프론트엔드 도메인만 접근 가능하도록 제한
- **express-validator**: 사용자 입력값 검증을 통한 유효성 보장
- **dotenv**: API 키 및 DB 정보 등 민감 정보를 외부에 노출하지 않도록 환경파일 분리

3.4 데이터베이스 설계

MySQL은 정형 데이터를 저장하며, 주요 테이블은 사용자 정보, 예약 내역, 공용 가전 사용 내역 등을 포함한다. Firebase Firestore는 실시간 데이터가 필요한 채팅, 상태 공유, 알림 등의 정보를 저장하며, Firebase Authentication 및 FCM과의 자연스러운 연동을 고려하였다.

3.5 실시간 기능

실시간 채팅과 상태 공유는 WebSocket과 Firebase Firestore를 병행하여 구현된다. 주요 기능은 다음과 같다.

기능	기술	설명
실시간 채팅	WebSocket, Firestore	빠른 메시지 송수신 및 히스토리 저장
알림 전송	Firebase Cloud Messaging	예약 알림, 미션 알림 등 푸시 알림 제공
상태 공유	Firestore Snapshot Listener	외출 여부 등 상태 변경을 실시간 반영

3.6 로그 및 에러 처리

Winston 모듈을 사용하여 요청 및 에러 로그를 파일 단위로 저장하며, 로그 레벨에 따라 서버 동작 상태를 구분 관리한다. 전체 API 요청에는 전역 에러 처리 미들웨어를 적용하여 예외 상황 발생 시 일관된 응답을 제공하도록 한다.

3.7 배포 및 운영

배포는 GitHub Actions와 AWS CodeDeploy를 통해 자동화되어 있으며, 백엔드는 AWS EC2 인스턴스에서 실행된다. 정적 파일은 Firebase Hosting을 통해 배포하며, 데이터베이스는 AWS RDS(MySQL)와 Firebase Firestore를 병행 사용한다. 개발과 운영 환경은 dotenv를 통해 .env.dev, .env.prod 등으로 구분되어 관리된다.

항목	기술	설명
서버	AWS EC2	Node.js 서버 호스팅
정적 파일	Firebase Hosting	Flutter 빌드 파일 배포
DB	AWS RDS, Firestore	데이터 유형에 따른 분산 저장
환경 변수	dotenv	환경별 설정 파일 분리 관리

3.8 프로젝트 구조

/src

```
├ /controllers          // 요청 핸들러
├ /routes               // API 라우터 정의
├ /services             // 비즈니스 로직
├ /middlewares          // 인증, 에러 처리 등 미들웨어
├ /models               // MySQL 기반 데이터 모델
├ /utils                // 공통 유틸 함수
└ /config               // DB 및 환경 설정
```

4 프론트엔드 기술 사양

4.1 기술 스택 개요

본 프로젝트의 프론트엔드는 Flutter 프레임워크를 기반으로 하여 iOS와 Android에서 모두 실행 가능한 **크로스플랫폼 앱**으로 구현되어 있다. 사용자 친화적인 UI/UX와 실시간 반응성, 푸시 알림 수신 등 다양한 사용자 인터랙션을 지원하기 위해 다음과 같은 기술 스택을 사용하였다.

항목	사용 기술	설명
프레임워크	Flutter	iOS/Android 동시 지원 앱 개발
상태 관리	provider	전체 앱 상태 및 위젯 재렌더링 관리
알림 수신	Firebase Cloud Messaging (FCM)	푸시 알림 수신 및 로직 처리
실시간 통신	Firebase Firestore, WebSocket	채팅 및 상태 공유 기능
인증	OAuth (Google, Kakao, Apple)	소셜 로그인 연동
이미지 처리	image_picker, Firebase Storage	이미지 선택 및 업로드
캘린더 UI	table_calendar	공동 일정 등록 및 시각화
룰렛 UI	flutter_spinbox	룰렛 인터페이스 구현

4.2 화면 구성 및 주요 기능

Flutter 앱은 주요 기능 단위로 화면이 구성되어 있으며, 각 기능은 REST API 또는 실시간 통신을 통해 백엔드와 연동된다. 다음은 주요 기능과 구현 기술에 대한 설명이다.

기능	설명	사용 기술
공동 일정 관리	청소/미션 일정 등록 및 편집	table_calendar, provider
방문객 예약	방문객 정보 입력 후 룸메이트 동의 요청	REST API, FCM
실시간 채팅	1:1 또는 그룹 채팅	WebSocket, Firestore
이미지 전송	채팅 또는 프로필 이미지 업로드	image_picker, Firebase Storage
프로필 설정	닉네임, 사진, 랜덤 생성 기능 포함	REST API, provider
룰렛 미니게임	청소 분담용 룰렛 실행	flutter_spinbox
소음 알림	귀여운 이모티콘 메시지로 전달	실시간 메시지 기반
피드백 시스템	이모티콘 기반 피드백 제출	provider, API
가전제품 예약	세탁기, 욕실 등 예약 및 충돌 방지	table_calendar, API
상태 공유	외출 여부, 재실 인원 공유	Firestore Snapshot Listener

4.3 상태 관리 방식

전역 상태 관리는 provider 패키지를 중심으로 구성되어 있으며, ChangeNotifier를 기반으로 각 기능별 Provider 클래스를 정의하였다. 상태 변화가 감지되면 관련 위젯이 자동으로 갱신되도록 구성되어 있다.

```
class ScheduleProvider with ChangeNotifier {
  List<Event> _events = [];

  void addEvent(Event event) {
```

```
      _events.add(event);
      notifyListeners();
    }
}
```

Provider는 일정 변경, 프로필 수정, 채팅 메시지 갱신 등에서 공통적으로 사용된다.

4.4 OAuth 인증 연동

소셜 로그인을 위해 Google, Kakao, Apple 계정을 지원하며, Flutter에서 발급한 accessToken을 백엔드로 전달한 후, 자체 JWT 토큰을 발급받아 앱 내 모든 요청에 포함한다.

동작 흐름은 다음과 같다:

1. 사용자가 Flutter 앱에서 소셜 계정으로 로그인
2. accessToken을 백엔드로 전달
3. 백엔드는 사용자 검증 후 JWT 발급
4. 클라이언트는 이후 모든 요청에 JWT 포함

4.5. 알림 시스템

Firebase Cloud Messaging(FCM)을 통해 예약, 일정, 룰렛 결과 등의 알림이 사용자 기기에 푸시된다. 알림 수신은 포그라운드 및 백그라운드 모두 지원하며, 알림 클릭 시 관련 화면으로 라우팅된다.

```
FirebaseMessaging.onMessage.listen((RemoteMessage message) {
  // 포그라운드 알림 수신 처리
});
```

4.6. 실시간 통신

채팅과 상태 공유는 실시간 데이터 반영이 중요한 기능으로, 다음과 같은 방식으로 구현되었다.

기능	방식	설명
채팅	WebSocket + Firestore	메시지 송수신 및 기록 저장
상태 공	Firestore Snapshot Listener	현재 외출 여부 등 실시간

유		표시

4.7. 디렉토리 구조 예시

전체 Flutter 프로젝트는 역할 단위로 디렉토리를 구분하여 유지보수가 용이하도록 설계되어 있다.

```
/lib
  ├ /screens          // 화면별 위젯 페이지
  ├ /widgets          // 재사용 가능 공통 위젯
  ├ /providers        // 상태 관리 클래스
  ├ /services         // API 통신 및 인증 로직
  ├ /models           // 데이터 모델 정의
  ├ /utils            // 공통 유틸 함수
  └ main.dart         // 앱 진입점
```

5 마무리

본 기술 사양서는 프로젝트의 기획 의도와 기술적 구현 방식을 명확히 정리하고, 팀 내 개발자 간의 원활한 협업과 유지보수의 효율성을 확보하기 위해 작성되었다. 백엔드는 안정성과 확장성을 고려한 구조로 설계되었으며, 프론트엔드는 직관적인 사용자 경험을 기반으로 다양한 기능을 통합하였다.

이 문서를 기반으로 향후 기능 개발, 테스트, 운영 과정에서의 일관된 기준을 유지하고자 하며, 프로젝트 완료 후에는 실제 운영 경험을 토대로 개선 사항을 도출하고 기술적 완성도를 더욱 높여나갈 예정이다.